

Overview of Data Structure 线性/数据结构分别有哪些/使用

Programming with **Linear** collections: Lists, Sets, Bags, Maps, Stacks, Queues, Priority Queues

Using **Linear** collections: • Programming with collections • Searching & Sorting Data • Implementing linear collections • Implementing sorting algorithms • Linked data structures and “pointers”

Programming with **Hierarchical** collections: Trees, binary trees, general trees

• Using tree structured collections: Building, Searching, Traversing tree structures • Implementing tree structured collections • Implementing linear collections : with binary search trees

Programming with Libraries 库结构, JAVA 的几个常用库

原因: Modern programs (especially GUI and network) are **too big** to build from scratch.

Have to **reuse code** written by other people: **Libraries** are collections of code designed for reuse.

Java API: a huge collection of **standard Libraries**. (指 java 标准类库提供的函数) 库中有很多 packages, packages 中有很多 classes.

Libraries to use: 1. **java.util:** **Collection** classes or Other **utility** classes. 2. **java.io:** Classes for **input and output**. 3.

javax.swing/java.awt: Large library of classes for **GUI** programs (图形用户界面程序)

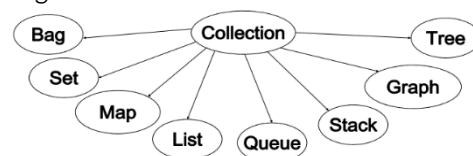
Using Libraries: 1. Read the documentation to pick useful library 2. import the package or class into your program 3. Read the documentation to identify how to use: **Constructors** for making instances, **Methods** to call, **Interfaces** to implement. 4. Use the classes as if they were part of your program

```
import java.util.*;
import java.io.*;
```

“Standard” Collections 标准的集群有哪些

Each of these is a **different type** of collection

- values **organised/structured** differently
- different constraints on **duplicates** 是否重复, and on **access** 访问顺序
- very **abstract** – don't care what type the elements are, don't care how they're stored or manipulated inside



Abstract Data Type 抽象数据类型: an **ADT** is a type of data, described at an abstract level:

- A new set contains no values.
- A set will contain a value **iff**
 - the value has been added to the set and
 - it has not been removed since adding it.
- A set will not contain a value **iff**
 - the value has never been added to the set, or
 - it has been removed from the set and has not been added since it was removed.

Operations: **add(value)**, **remove(value)**, **contains(value) → boolean**

- Specifies the **operations** that can be done to an object of this type
- Specifies how it will **behave**.

Java Interfaces: 一个 JAVA 接口对应一个 ADT

- Specifies what **methods** can be called on objects of this type (specifies name, parameters and types, and type of return value)
- **Behaviour** of methods is only given in comments (but cannot be enforced)

```
public interface Set {
    public void add(??? item); /* ...description... */
    public void remove(??? item); /* ...description... */
    public boolean contains(??? item); /* ...description... */
    ...
    // (plus lots more methods in the Java Set interface)
```

× No constructors

× No fields

× No method bodies.

- can't make an instance: **new Set()**

- doesn't say how to store the data

- doesn't say how to perform the operations

Parameterised Types 参数化类型

- Interfaces may have type parameters (eg, type of the element):

```
public interface Set<T> {
    public void add(T item); /* ...description... */
    public void remove(T item); /* ...description... */
    public boolean contains(T item); /* ...description... */
    ... // (lots more methods in the Java Set interface)
```

It's a Set of something, as yet unspecified

- When declaring variable, specify the actual type of element

```
private Set<Person> friends;
private List<Shape> drawing;
```

Collection Type

Type of value in Collection

一个集群的结构和访问规则不随其中值的类型的改变而改变

The structure and access discipline of a collection is the same, regardless of the type of value in it:

- A set of Strings, a set of Persons, a set of Shapes, a set of integers all behave the same way.

一个集群只有一个接口

Only want one Interface for each kind of collection. (there is only one **Set** interface)

要指定群集中的种类: 接口和类被参数化

- Interface has a **type parameter**
- When declaring a variable collection, you specify
 - ① the type of the collection and
 - ② the type of the elements of the collection

Interfaces:

- **Collection**
 - = Bag (most general)
- **List**
 - = ordered collection
- **Set**
 - = unordered, no duplicates
- **Queue**
 - ordered collection, limited access (add at one end, remove from other)
- **Map**
 - = key-value pairs (or mapping)
- List classes:
 - ArrayList, LinkedList, vector...
- Set classes:
 - HashSet, TreeSet, ...
- Map classes:
 - HashMap, TreeMap, ...

Using the Java Collection Library: Class (接口没有 constructors, 需要 class 来 create instances)

Classes in the Java Collection Library **implement** the interfaces: 1. **Define constructors** to construct new instances

2. **Define method bodies** for performing the operations

3. **Define fields** to store the values

```
private List<Shape> drawing = new ArrayList<Shape> ();
```

```
Set<Person> friends = new HashSet<Person> ();
```

ArrayList

predefined class

stores a list of items: a collection of items kept in a particular order

part of the java.util package ⇒ need to import java.util.*; at head of file

Using ArrayList: methods

- ArrayList has many methods!, including:
 - size(): returns the number of items in the list
 - add(item): adds an item to the end of the list
 - add(index, item): inserts an item at index (relocates later items)
 - set(index, item): replaces the item at index with item
 - contains(item): true if the list contains an item that equals item
 - get(index): returns the item at position index
 - remove(item): removes an occurrence of item (what if there are duplicates in the ArrayList?)
 - remove(index): removes the item at position index (both relocate later items)
- You can use the "for each" loop on an array list, as well as a for loop

Using ArrayList

```
private ArrayList<Student> students = new ArrayList<Student>();
Student s = new Student("Lindsay King", "300012345")
students.add(s);
students.add(0, new Student(fscanner));

for (int i = 0; i < students.size(); i++)
    System.out.println(students.get(i).toString());

for (Student st : students)
    System.out.println(st.toString());

if (students.contains(current)){
    file.println(current);
    students.remove(current);
}
```

declaring

```
private ArrayList<Student> students = new ArrayList<Student>();
```

- The type of values in the list is between "<" and ">" after ArrayList.
- No maximum; no initial size; no explicit count

Q&A

What is a **software "library"**?

a library is a collection of pre-written code or software modules that can be reused by developers in their own software projects

Define Java "Package".

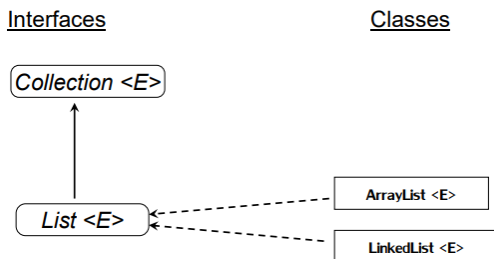
a package is a way of organizing related classes and interfaces together

What is the Java statement to **include package or class** into your program?

```
import and package package com.example.mypackage;
import java.util.ArrayList;
```

Interfaces can extend other interfaces 语句: interface List extends Collection

The **sub interface** has all the methods of the **super interface** plus its own methods



- Collection<E>
 - isEmpty() → boolean
 - size() → int
 - contains(E elem) → boolean (whether it succeeded)
 - add(E elem) → boolean (whether it removed an item)
 - remove(E elem) → boolean
 - iterator() → iterator<E>
- List<E>
 - add(int index, E elem) → E (returns the item removed)
 - remove(int index) → E
 - get(int index) → E (returns the item replaced)
 - set(int index, E elem) → E
 - indexOf(E elem) → int
 - subList(int from, int to) → List<E>

Example: 分清接口和类/声明创造

ToDoList – collection of **tasks**, in order they should be done.
Collection type: **List** of tasks

```
class
interface
public class ToDoList implements ActionListener{
    :
    private List<Task> tasks;
    :
    /* read list of tasks from a file, */ 实现
    public void readTasks(String fname){
        try {
```

iterators

```
public void displayTasks(){
    textArea.setText(tasks.size() + " tasks to be done:\n");
    for (Task task : tasks){
        textArea.append(task + "\n");
    }
}

for (int i=0; i<tasks.size(); i++){
    textArea.append(tasks.get(i) + "\n");
}

Or

iterator<Task> iter = tasks.iterator();
while (iter.hasNext()){
    textArea.append(iter.next() + "\n");
}
```

Declaring 和 Constructing

Variable or field declared to be of the interface type

- Specify the type of the collection
- Specify the type of the value

```
private List<Task> tasks;
```

Create an object of a class that implements the type:

- Specify the class
 - Specify the type of the value
- ```
tasks = new ArrayList<Task> ();
```

## 迭代器接口

**Scanner** 是一个奇特的迭代器  
所有 **Collection** 类都可以使用  
迭代器

Iterator is an interface:

```
public interface Iterator<E> {
 public boolean hasNext();
 public E next();
}
```

## Bags

There are **no standard** implementations Bag

Collection<E> = Bag (most general)

A Bag is a collection with

- no structure or order maintained
- no access constraints (access any item any time)
- duplicates allowed

Minimal Operations:

- add(value) → returns true iff a collection was changed
- remove(value) → returns true iff a collection was changed
- contains(value) → returns true iff value is in bag uses equal to test.
- findElement(value) → returns a matching item, iff in bag

Plus

- size(), isEmpty(), iterator(), clear(), addAll(collection), removeAll(collection), containsAll(collection), ...

When to use a Bag? 适用

- When there is no need to order a collection, and duplicates are possible:
  - A collection of current logged-on users (can there be dups?)
  - The books in a book collection (can there be dups?)

## Set

Set is a collection with:

- no structure or order maintained
- no access constraints (access any item any time)
- Only property is that duplicates are excluded

Operations:

- (Same as Bag, but different behaviour)
- add(value) → true iff value was added (ie, no dupl)
  - remove(value) → true iff value removed (was in set)
  - contains(value) → true iff value is in the set
  - findElement(value) → matching item, iff value is in the set

Sets are as common as Lists

## Map

When declaring and constructing, must specify two

- Type of the key, and type of the value

```
private Map<String, Integer> phoneBook;
:
phoneBook = new HashMap<String, Integer>();
```

Central operations:

- get(key), → returns value associated with key (or null)
- put(key, value), → sets the value associated with key (and returns the old value, if any)
- remove(key), → removes the key and associated value (and returns the old value, if any)
- containsKey(key), → boolean
- size()

## Stack

```
public interface StackInterface
```

```
{
 /** Task: Adds a new entry to the top of the stack.
 * @param newEntry an object to be added to the stack */
 public void push(Object newEntry);

 /** Task: Removes and returns the top of the stack.
 * @return either the object at the top of the stack or null if the stack was empty */
 public Object pop();

 /** Task: Retrieves the top of the stack.
 * @return either the object at the top of the stack or null if the stack is empty */
 public Object peek();

 /** Task: Determines whether the stack is empty.
 * @return true if the stack is empty */
 public boolean isEmpty();

 /** Task: Removes all entries from the stack */
 public void clear();
} // end StackInterface
```

Read and push onto a stack

Pop them off the stack

```
public void reverseNums(Scanner sc){
 Stack<Integer> myNums = new ArrayStack<Integer>();
 while (sc.hasNext()){
 myNums.push(sc.nextInt());
 }
 while (!myNums.isEmpty())
 textArea.append(myNums.pop() + "\n");
}
```

There exists a **Stack interface** and different implementations of it (ArrayStack, LinkedStack, etc)

- When a method is called
  - Runtime environment creates activation **record**
  - Shows **method's state** during execution

- **Ordinary queues:** only add at the back
 
- **Priority queues:** add or remove with a given priority
 
- Used for
 
  - Operating Systems, Network Applications, Multi-user Systems
    - Handling requests/events/jobs that must be done in order
    - (memory pool holding such requests are often called a "buffer" in this context)
  - Simulation programs
    - Representing queues in the real world (traffic, customers, deliveries, ...)
    - Managing events that must happen in the future
  - Search Algorithms
    - Computer Games
    - Artificial Intelligence
- Java provides
  - a **Queue interface**
  - several classes: **LinkedList**, **PriorityQueue**

Program                      Program stack at three points in time (PC is the program counter)

```
graph TD
 subgraph main_scope [main]
 direction TB
 PC1[PC = 1]
 arg1[arg = 800]
 end

 subgraph methodA_scope [methodA]
 direction TB
 PC2[PC = 100]
 a1[a = 5]
 end

 subgraph methodB_scope [methodB]
 direction TB
 PC3[PC = 150]
 b1[b = 2]
 end

 subgraph methodA_scope2 [methodA]
 direction TB
 PC4[PC = 120]
 a2[a = 5]
 z1[z = 2]
 end

 subgraph main_scope2 [main]
 direction TB
 PC5[PC = 50]
 arg2[arg = 800]
 x1[x = 5]
 y1[y = 0]
 end

 subgraph methodA_scope3 [methodA]
 direction TB
 PC6[PC = 150]
 b2[b = 2]
 end

 subgraph methodA_scope4 [methodA]
 direction TB
 PC7[PC = 120]
 a3[a = 5]
 z2[z = 2]
 end

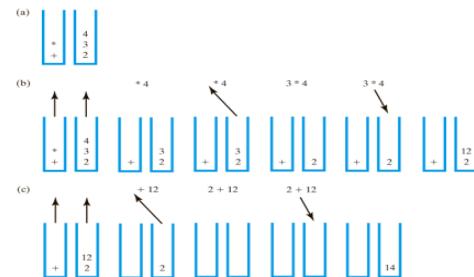
 subgraph main_scope3 [main]
 direction TB
 PC8[PC = 50]
 arg3[arg = 800]
 x2[x = 5]
 y2[y = 0]
 end
```

The diagram illustrates the process of finding a variable's value when it is used in a function call. It shows three main scenarios:

- Scenario 1:** A function call `methodA()` from `main`. The variable `a` is found in the `methodA` function's local scope.
- Scenario 2:** A function call `methodB()` from `main`. The variable `b` is found in the `methodB` function's local scope.
- Scenario 3:** A function call `methodA()` from `main`. The variable `a` is found in the `methodA` function's local scope.

In all cases, the variable is found in the function's local scope. The diagram also shows that the variable `a` is not found in the `main` scope when it is used in the `methodA` function.

### Evaluating Infix Expressions using Two Stacks



### Example of Map

```
private Map<String, String> movieCast; // character → actor
:
public void lookup(){
 String name = askName("Character to look up");
 if (movieCast.containsKey(name))
 textArea.setText(name + " : " + movieCast.get(name));
 else
 textArea.setText("No entry for " + name);
}

public void update(){
 String name = askName("Character to update");
 String actor = askName("Actor who played " + name);
 String old = movieCast.put(name, actor);
 if (old==null)
 textArea.setText(" added " + name + " played by " + actor);
 else
 textArea.setText(" replaced " + old + " by " + actor + " for " + name);
}
```

集群拓展了迭代器接口：可以访问集群内部，一次提供一个元素，每个集群都需要一个相关的 Iterator class

```
for (String s : list) {
 // 等价于 for (Iterator i=list.iterator(); i.hasNext();) i.next();
}
```



- E will be bound to a real type when a List is declared or constructed.



## an Abstract class

- Defines the complex methods in terms of the basic methods
- Leaves the basic methods "abstract" (no body)
- classes implementing List can extend the abstract class.

|                                                                                                                                                                                     |                                                                                                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Interface</b> <ul style="list-style-type: none"> <li>specifies type</li> <li>defines method headers</li> </ul>                                                                   | <b>List &lt;E&gt;</b> <ul style="list-style-type: none"> <li>Specifies sequence of E type</li> </ul>                                                                                                                              |
| <b>Abstract Class</b> <ul style="list-style-type: none"> <li>implements Interface</li> <li>defines some methods</li> <li>leaves other methods "abstract"</li> </ul>                 | <b>AbstractList &lt;E&gt;</b> <ul style="list-style-type: none"> <li>implements List &lt;E&gt;</li> <li>defines array of &lt;E&gt;</li> <li>defines addAll, subList, ...</li> <li>add, set, get, ... are left abstract</li> </ul> |
| <b>Class</b> <ul style="list-style-type: none"> <li>extends Abstract Class</li> <li>defines data structures</li> <li>defines basic methods</li> <li>defines constructors</li> </ul> | <b>ArrayList &lt;E&gt;</b> <ul style="list-style-type: none"> <li>extends AbstractList</li> <li>implements fields &amp; constructor</li> <li>implements add, get, ...</li> </ul>                                                  |

```
public abstract class AbstractList<E> implements List<E>{
```

No constructor or fields

```
public abstract int size();
```

declared abstract - must be defined in a real class

```
public boolean isEmpty(){
```

defined in terms of size()

```
return (size() == 0);
```

```
}
```

```
public abstract E get(int index);
```

declared abstract - must be defined in a real class

```
public void add(int index, E element){
```

defined to throw exception should be defined in a real class

```
throws new UnsupportedOperationException();
```

```
}
```

```
public boolean add(E element){
```

defined in terms of other add

```
add(size(), element);
```

```
}
```

## ArrayList

```
public class ArrayList<E> extends AbstractList<E>{
```

// fields to hold the data:

need an array for the items and an int for the count.

// constructor(s)

Initialise the array

// definitions of basic methods not defined in AbstractList

```
size()
```

```
get(index)
```

```
set(index, value)
```

```
remove(index)
```

```
add(index, value)
```

```
iterator()
```

Can give other methods to override the inherited methods, if it would be more efficient

// (other methods are inherited from AbstractList)

// definition of the iterator class

## ArrayList: fields and constructor

```
public class ArrayList<E> extends AbstractList<E> {
 private E[] data;
 private int count=0;
 private static int INITIALCAPACITY = 16;
```

```
public ArrayList(){
 data = (E[]) new Object[INITIALCAPACITY];
}
```

Can't use type variables as array constructors!!!!

Must Create as `Object[]` and cast to `E[]`

The compiler will return a warning!

\* " .... uses unchecked or unsafe operations" (why it is 'unchecked?')

## ArrayList: size, isEmpty, get

```
/** Returns number of elements in collection as integer */
public int size() {
 return count;
}
```

```
/** Returns true if this set contains no elements. */
public boolean isEmpty(){
 return count==0;
}
```

```
public E get(int index){
 if (index < 0 || index >= count)
 throw new IndexOutOfBoundsException();
 return data[index];
}
```

## set

```
public E set(int index, E value){
 if (index < 0 || index >= count)
 throw new IndexOutOfBoundsException();
 E ans = data[index];
 data[index] = value;
 return ans;
}
```

## remove

```
public E remove(int index){
 if (index < 0 || index >= count)
 throw new IndexOutOfBoundsException();
 E ans = data[index];
 for (int i=index; i < count; i++)
 data[i]=data[i+1];
 count--;
 return ans;
}
```

## remove (fixed)

```
/** Removes the element at the specified index, and returns it.
 * Throws an IndexOutOfBoundsException if index is out of
 * bounds */
public E remove(int index){
 if (index < 0 || index >= count)
 throw new IndexOutOfBoundsException();
 E ans = data[index];
 for (int i=index+1; i < count; i++)
 data[i-1]=data[i];
 count--;
 data[count] = null;
 return ans;
}
```

## ensureCapacity

```
private void ensureCapacity () {
 if (count < data.length) return;
 E[] newArray = (E[]) (new Object[data.length+INITIALCAPACITY]);
 for (int i = 0; i < count; i++)
 newArray[i] = data[i];
 data = newArray;
}

OR

private void ensureCapacity () {
 if (count < data.length) return;
 E[] newArray = (E[]) (new Object[data.length * 2]);
 for (int i = 0; i < count; i++)
 covv to new array
}
```

## add

```
/** Adds the specified element at the specified index */
public void add(int index, E item){
 if (index < 0 || index >= count)
 throw new IndexOutOfBoundsException();
 for (int i=count; i > index; i--)
 data[i]=data[i-1];
 data[index]=item;
 count++;
}
```

## add (fixed) (at i)

```
/** Adds the specified element at the specified index */
public void add(int index, E item){
 if (index < 0 || index >= count)
 throw new IndexOutOfBoundsException();
 ensureCapacity();
 for (int i=count; i > index; i--)
 data[i]=data[i-1];
 data[index]=item;
 count++;
}
```

## add(at end)

```
public void add (E item){
 ensureCapacity();
 data[count++] = item;
}
```

## iterator

```
/** Returns an iterator over the elements in the List */
public Iterator<E> iterator(){
 return new ArrayListIterator<E>(this);
}

private class ArrayListIterator<E> implements Iterator<E>{
 private ArrayList<E> list; // reference to the list it is iterating down
 private int nextIndex = 0; // the index of the next value to return
 private boolean canRemove = false; // to disallow the remove operation initially

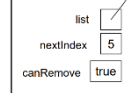
 /** Constructor */
 private ArrayListIterator(ArrayList<E> list) {
 this.list = list;
 }

 /** Return true if iterator has at least one more element */
 public boolean hasNext() {
 return (nextIndex < list.count);
 }
}
```

## iterator ( next, no remove )

```
/** Return next element in the List */
public E next() {
 if (nextIndex >= list.count) throw new NoSuchElementException();
 return list.get(nextIndex++);
}
```

```
/** Remove from the list the last element returned by the iterator.
 * Can only be called once per call to next. */
public void remove(){
 throw new UnsupportedOperationException();
}
```



## iterator ( next, with remove )

```
/** Return next element in the List */
public E next() {
 if (nextIndex >= list.count) throw new
 NoSuchElementException();
 canRemove = true;
 return list.get(nextIndex++);
}

/** Remove from the list the last element returned by the iterator.
 * Can only be called once per call to next. */
public void remove(){
 if (!canRemove) throw new IllegalStateException();
 canRemove = false;
 nextIndex--;
 list.remove(nextIndex);
}
```

what if we don't put counter back to last item?

After removal, nextIndex will be pointing at which item?

## determine the costs of a program

**Time:** 1.Run the program and count the milliseconds/minutes/days. 2.Count the number of steps/operations the algorithm will take.

**Space:** 1.Measure the amount of memory the program occupies 2.Count the number of elementary data items the algorithm stores.

- programs: benchmarking
- algorithms: analysis

## Benchmarking

Measure:

- actual programs
- on real machines
- on specific input
- measure elapsed time
  - System.currentTimeMillis()
    - time from system clock in milliseconds (long)
- measure real memory usage

Problems:

- what input
  - choose test sets careful
  - use large data sets
  - don't include user input
- other users/processes
  - minimise
  - average over many runs
- which computer?
  - soecifv details

## Algorithm complexity

Abstract away from the details of

- the hardware
- the operating system
- the programming language
- the compiler
- the program
- the specific input

Measure number of "steps" as a function of the data size.

- worst case (easier)
- average case (harder)
- best case (easy, but useless)

Construct an expression for the number of steps:

- cost =  $3.47n^2 - 67n + 53$  steps
- cost =  $3n \log(n) - 5n + 6$  steps
- simplified into terms of different powers/functions of n

## Big Oh Notation

A notation for describing efficiency of computer algorithms:

assuming a 100-MHz clock,  $N = 1024k = 2^{20}$   
 $100 \times 10^6 \text{ Hz} \rightarrow 10^8 \text{ s} = 10 \text{ ns}$

$O(1)$  - constant time, 10 ns

$O(\log N)$  - logarithmic time, 200 ns

$O(N)$  - linear time, 10.5ms

$O(N \log N)$  -  $n \log n$  time, 210 ms

$O(N^2)$  - quadratic time, 3.05 hours

$O(N^3)$  - cubic time, 365 years

$O(2^N)$  - exponential,  $10^{(10^5)}$  years

Usually we are only interested in the **asymptotic** time complexity, i.e., when  $n$  is large

$O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

## recursive functions

A recursive function is a function that calls itself directly or indirectly

Related to recursive problem solving: binary tree traversal (**divide & conquer**)

The function knows how to solve a base case (**stopping rule**)

A recursion step is needed to divide the problem into sub-problems (**key step**)

Need to check for **termination**

```
long fibonacci(long n)
{
 if (n == 0 || n == 1)
 return n;
 else
 return fibonacci(n - 1) + fibonacci(n - 2);
}

long fac(long number)
{
 if (number <= 1) base case
 return 1;
 else
 return (number * fac(number - 1)); recursive step
}
```

## Testing Collection Implementations

### 集群实现测试

Write a **test method**

- As part of the class or as a separate testing class
- Should test all the operations
- Should test normal and extreme cases

Good practice:

- write it first (**black box testing**)
- implement the collection
- extend the test method to cover the special cases of the implementation (**white box testing**)

Nicer design uses **tests/assertions**:

- check that the code does the right thing
- only report when there is a problem or error

May take longer to write than the collection code!

```
public class ListNode<E> {
 private E value;
 private ListNode<E> next;
 public ListNode(E item, ListNode<E> nextNode) {
 value = item;
 next = nextNode;
 }
 public E get() { return value; }
 public ListNode<E> next() { return next; }
 public void set(E item) {
 value = item;
 }
 public void setNext(ListNode<E> nextNode) {
 next = nextNode;
 }
}
```

### Linked List Structures

- Can be drawn with Nodes inside Nodes:



- "Pointers" are better:

- Each node contains a reference to the next node
- reference = memory location of / pointer to object



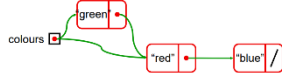
- Can view a node in two ways:

- an object containing two fields
- the head of a linked list of values

```
ListNode<String> colours = new ListNode<String>("red", null);
```

```
colours.setNext(new ListNode<String>("blue", null));
```

```
colours = new ListNode<String>("green", colours);
```



```
System.out.format("1st: %s\n", colours.get()); // green
System.out.format("2nd: %s\n", colours.next().get()); // red
System.out.format("3rd: %s\n", colours.next().next().get()); // blue
```

## Memory allocation 分配内存

- What are references/pointers?

- Pointers/references are an address or a chunk of memory, where data can be stored.

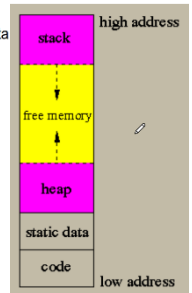
- How do you get this memory allocated?

- You've been doing it using **new**:

- creating an object will allocate some **heap** memory for the object.
- new** returns the address of the chunk of memory
- copying the address does not copy the chunk of memory.

- Memory from the heap must be recycled after use:

- The **garbage collector** automatically frees up any memory chunks that no longer have anything pointing/referring to them.
- This frees you from having to worry about explicitly freeing memory.

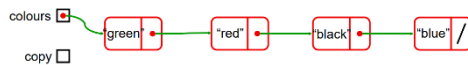


### green-red-black-blue

```
ListNode<String> colours = new ListNode<String>("red", null);
colours.next = new ListNode<String>("blue", null);
colours = new ListNode<String>("green", colours);
colours.next.next = new ListNode<String>("black", colours.next.next);
```

Remove the third node:

```
colours.next.next = colours.next.next.next;
```



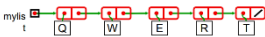
## Inserting

```
/** Insert the value at position n in the list (counting from 0)
 * Assumes list is not empty, n>0, and n <= length of list */
```

```
public void insert(E item, int n, ListNode<E> list) {
 if (n == 1)
 list.next = new ListNode<E>(item, list.next);
 else
 insert(item, n-1, list.next);
}
```

or

```
public void insert(E item, int n, ListNode<E> list) {
 int pos = 0;
 ListNode<E> rest = list; // rest is the pos'th node
 while (pos < n-1) {
 pos++;
 rest = rest.next;
 }
 rest.next = new ListNode<E>(item, rest.next);
}
```



## Removing

```
/** Remove the value from the list
 * Assumes list is not empty, and value not in first node */
```

```
public void remove(E item, ListNode<E> list) {
 if (list.next == null) return; // we are at the end of the list
 if (list.next.value.equals(item))
 list.next = list.next.next;
 else
 remove(item, list.next);
}
```

or

```
public void remove(E item, ListNode<E> list) {
 ListNode<E> rest = list;
 while (rest.next != null && !rest.next.value.equals(item))
 rest = rest.next;
 if (rest.next != null)
 rest.next = rest.next.next;
}
```

Why have a 'rest' to hold 'list'?

## List using linked nodes with header

### LinkedList

- LinkedList extends **AbstractList**
- Has fields for linked list of **Nodes** and **count**
- Has an **inner class**: **Node**, with public fields
- get(index)**, **set(index, item)**,
  - loop to index'th node, then get or set value
- add(index, item)**, **remove(index)**:
  - deal with special case of **index == 0**
  - loop along list to node **one before** index'th node (Why?), then add or remove
  - check if go past end of list **越过头**
- remove(item)**,
  - deal with special case of item in first node (i.e. conversion into empty set after removal)
  - loop along list to node one before node containing item (Why?), then remove
  - check if go past end of list

### constructors

```
public class LinkedList<E> extends AbstractList<E> {
 private Node<E> data;
 private int count;
 public LinkedList() {
 data = null;
 count = 0;
 }
 /** Inner class: Node */
 private class Node<E> {
 public E value;
 public Node<E> next;
 public Node(E val, Node<E> node) {
 value = val;
 next = node;
 }
 }
}
```

### get

```
public E get(int index) {
 if (index < 0) throw new IndexOutOfBoundsException();
 Node<E> node = data;
 int i = 0; // position of node
 while (node != null && i++ < index) node = node.next;
 if (node == null) throw new IndexOutOfBoundsException();
 return node.value;
}
```

### Set

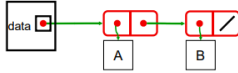
```
public E set(int index, E value) {
 if (index < 0) throw new IndexOutOfBoundsException();
 Node<E> node = data;
 int i = 0; // position of node
 while (node != null && i++ < index) node = node.next;
 if (node == null) throw new IndexOutOfBoundsException();
 E ans = node.value;
 node.value = value;
 return ans;
}
```

Same as get



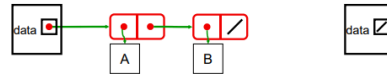
## add

```
public void add(int index, E item){
 if (item == null) throw new IllegalArgumentException();
 if (index==0){ // add at the front.
 data = new Node(item, data);
 count++;
 return;
 }
 Node<E> node=data;
 int i = 1; // position of next node
 while (node!=null && i++ < index) node=node.next;
 if (node == null) throw new IndexOutOfBoundsException();
 node.next = new Node(item, node.next);
 count++;
 return;
}
```



## remove

```
public boolean remove (Object item){
 if (item==null || data==null) return false;
 if (item.equals(data.value)) // remove the front item.
 data = data.next;
 else { // find the node just before a node containing the item
 Node<E> node = data;
 while (node.next!=null && !node.next.value.equals(item))
 node=node.next;
 if (node.next==null) return false; // off the end
 node.next = node.next.next; // splice the node out of the list
 }
 count--;
 return true;
}
```



## Linked Stacks

Make the top of the Stack the front of the list.



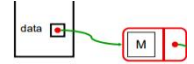
pop()  
push("A")

## 实现

```
public class LinkedStack <E> extends AbstractCollection <E> {
 private Node<E> data = null;
 public LinkedStack() {}
 public int size(){...}
 public boolean isEmpty(){...}
 public E peek(){...}
 public E pop(){...}
 public void push(E item){...}
 public Iterator <E> iterator(){...}
}
```

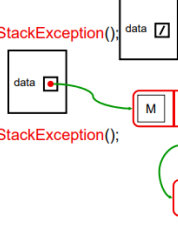
## isEmpty/size

```
public boolean isEmpty(){
 return data==null;
}
public int size () {
 if (data == null) return 0;
 else return data.size();
}
// Need size() method in Node class:
public int size () {
 int ans = 0;
 for (Node<E> rest = data; rest!=null; rest=rest.next)
 ans++;
 return ans;
}
```



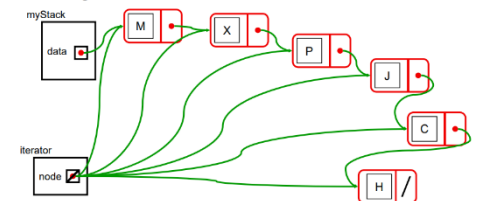
## peek/pop/push/iterator

```
public E peek(){
 if (data==null) throw new EmptyStackException();
 return data.value;
}
public E pop(){
 if (data==null) throw new EmptyStackException();
 E ans = data.value;
 data = data.next;
 return ans;
}
public void push(E item){
 if (item == null) throw new IllegalArgumentException();
 data = new Node(item, data);
}
public Iterator <E> iterator(){
 return new NodeIterator(data);
}
```



```
private class NodeIterator <E> implements Iterator <E>{
 private Node<E> node; // node containing next item
 public NodeIterator (Node <E> node) {
 this.node = node;
 }
 public boolean hasNext () {
 return (node != null);
 }
 public E next () {
 if (node==null) throw new NoSuchElementException();
 E ans = node.get();
 node = node.next();
 return ans;
 }
 public void remove(){
 throw new UnsupportedOperationException();
 }
}
```

Iterating down a stack.



```
for (String str: myStack)
 System.out.println(str)
```

## Linked Queue

Application of Queues  
user job queue

print spooling queue

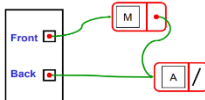
I/O event queue

incoming packet queue

outgoing packet queue

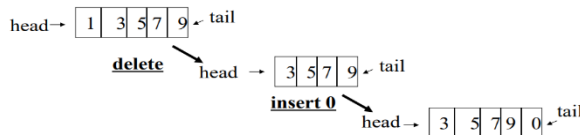
## 实现

```
public class LinkedQueue <E> implements AbstractQueue <E> {
 private Node<E> front = null;
 private Node<E> back = null;
 public LinkedQueue() {}
 public int size(){...}
 public boolean isEmpty(){...}
 public E peek(){...}
 public E poll(){...}
 public void offer(E item){...}
 public Iterator <E> iterator(){...}
}
```



Example: waiting lines

Insertion at the end (tail), deletion from the front (head)



## peek/poll/offer

```
public E peek(){
 if (front==null) return null;
 else return front.value;
}
public E poll(){
 if (front==null) return null;
 E ans = front.value;
 front = front.next;
 if (front==null) back = null;
 return ans;
}
public boolean offer(E item){
 if (item == null) return false;
 if (front == null){
 back = new Node(item, null);
 front = back;
 }
 else {
 back.next = (new Node(item, null));
 back = back.next;
 }
 return true;
}
```

## Linked Stack and Queue 总结

Uses a "header node"

- contains link to head node, and maybe last node of linked list

Important to choose the right end.

- easy to add or remove from head of a linked list
- hard to add or remove from the last node of a linked list
- easy to add to last node of linked list if have pointer to tail

Linked Stack and Queue:

- all main operations are  $O(1)$

Can combine Stack and Queue

- addFirst, addLast, removeFirst
- also need removeLast to make a "Deque" (double-ended queue)  $\Rightarrow$  need doubly linked list (why?)
- See the java "LinkedList" class.

## ArraySet

Order is not significant

- ⇒ add() can choose to put a new item anywhere. where?
- ⇒ can reorder when removing an item. how?

Duplicates not allowed.

- ⇒ must check if item already present before adding

Contains(value):  $O(n)$

search through array,  
if value equals item  
return true  
return false

Add(value):  $O(n)$

if not contains(value),  
place value at end, (doubling array if necessary)  
increment size

Remove(value):  $O(n)$

search through array  
if value equals item  
replace item by item at end. (why?)  
decrement size  
return

## 构造 sortedArraySet

Sort the items all at once

```
public SortedArraySet(Collection<E> col){
 // Make space
 count=col.size();
 data = (E[]) new Object[count];
 // Put items from collection into the data array.
 col.toArray(data);
 // sort the data array.
 Arrays.sort(data);
}
```

## Making ArraySet faster: sorted

Binary Search: Finding "Eel"

|     |     |     |     |     |     |     |     |   |  |
|-----|-----|-----|-----|-----|-----|-----|-----|---|--|
| 8   |     |     |     |     |     |     |     |   |  |
| Ant | Bee | Cat | Dog | Eel | Fos | Gnu | Pig |   |  |
| 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8 |  |

If the items are sorted ("ordered"), then we can search for:

- Look in the middle:
  - if item is middle item ⇒ return
  - if item is before middle item ⇒ look in left half
  - if item is after middle item ⇒ look in right half

如果 ArraySet 是排序好的, 用二元搜索, 则算法复杂度为  $\log(n)$   
SortedArraySet: with Binary Search

- Binary Search is fast:  $O(\log(n))$ 
  - contains:  $O(\log(n))$
  - add:  $O(n)$
  - remove:  $O(n)$

## Binary search: $\log(n)$

```
private boolean contains(Object item){
 Comparable<E> value = (Comparable<E>) item;
 int low = 0; // min possible index of item
 int high = count-1; // max possible index of item (if present)
 while (low <= high){
 int mid = (low + high) / 2;
 int comp = value.compareTo(data[mid]);
 if (comp == 0) // item is present
 return true;
 if (comp < 0) // item in [low .. mid-1]
 high = mid - 1;
 else // item in [mid+1 .. high]
 low = mid + 1;
 }
 return false; // item in [low .. high] and low > high,
 // therefore item not present
}
```

If you have to call add() and/or remove() many items, then SortedArraySet is no better than ArraySet!

- Both  $O(n)$
- Either pay to search
- Or pay to keep it in order

If you only have to construct the set once, and then many calls to contains(), then SortedArraySet is much better than ArraySet.  
SortedArraySet contains() is  $O(\log(n))$

## 一次性构造所有 items

原因:

Constructing a sorted array one item at a time is very slow:

- add(item) is  $n/2$  on average  $[O(n)]$   
⇒  $1/2 + 2/2 + 3/2 + \dots + n/2$  is  $n^2/4$   $[O(n^2)]$
- ≈ 2,500,000,000,000,000 steps for 100,000,000 items  
⇒ 25,000,000 seconds = 289 days at 10ns per step.

There are sorting algorithms that are much faster if you can sort whole array at once:  $O(n \log(n))$

- ≈ 2,700,000,000 steps for 100,000,000 items  
⇒ 27 seconds at 10ns per step.

## Ways of sorting

- Selecting sorts:  $O(n^2)$ 
  - Find the next largest/smallest item and put in place
  - Builds the correct list in order
- Inserting Sorts:  $O(n^2)$ 
  - For each item, insert it into an ordered sublist
  - Builds a sorted list, but keeps changing it
- Compare and Swap Sorts:  $O(n^2)$  /  $O(n \log n)$ 
  - Find two items that are out of order, and swap them
  - Keeps "improving" the list
- Radix Sorts
  - Look at the item and work out where it should go.
  - Only works on some kinds of values.
- ...

## Implementing Sorting Algorithms

- Could sort Lists
  - ⇒ general and flexible
  - but efficiency depends on how the List is implemented
- Could sort Arrays.
  - ⇒ less general
  - but efficiency is well defined
  - easy to convert any Collection to an array: `toArray()` method.
- Comparing items:
  - require items to be comparable (natural order)
  - provide comparator (prescribed order)
  - handle both.

## Selection Sort

```
public void selectionSort(E[] data, int size, Comparator<E> comp){
 // for each position, from 0 up, find the next smallest item
 // and swap it into place
 for (int place=0; place<size-1; place++){
 int minIndex = place;
 for (int sweep=place+1; sweep<size; sweep++){
 if (comp.compare(data[sweep], data[minIndex]) < 0)
 minIndex=sweep;
 }
 swap(data, place, minIndex);
 }
}
```

- Efficiency
  - worst-case:  $O(n^2)$
  - average case exactly the same.
- Requirements on Data
  - Needs random-access data, but easy to modify for files
  - Needs compare and swap
- Space Usage
  - in-place

## bubbleSort

```
public void bubbleSort(E[] data, int size, Comparator<E> comp){
 // Repeatedly scan array, swapping adjacent items if out of order
 // Builds a sorted region from the end
 for (int top=size-1; top>0; top--){
 for (int sweep=0; sweep<top; sweep++){
 if (comp.compare(data[sweep], data[sweep+1]) > 0)
 swap(data, sweep, sweep+1);
 }
 }
}
```

Efficiency

- worst-case:  $O(n^2)$
- average case:  $O(n^2)$ , no better than worst

Requirements on Data

- Needs random-access data, but can modify for files
- Needs compare and swap

Space Usage

- in-place

## Slow Sorts

Insertion sort, Selection Sort, Bubble Sort:

- All slow (except Insertion sort on almost sorted lists)
- $O(n^2)$

Problem:

- Insertion and Bubble
  - only compare adjacent items
  - only move items one step at a time
- Selection
  - compares every pair of items – ignores results of previous comparisons.

Solution:

- compare and swap items at a distance
- do not perform redundant comparisons

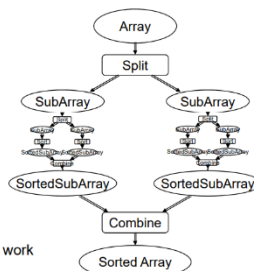
## Divide and Conquer Sorts(fast sort)

To Sort:

- Split
- Sort each part (recursive)
- Combine

Where does the work happen?

- MergeSort:
  - split trivial
  - combine does all the work
- QuickSort:
  - split does all the work
  - combine trivial



## MergeSort

given two sorted arrays, merge them into one sorted array  
keep track of the smallest element in each array, output the smaller of the two to a third array

Continue until both arrays are exhausted

If any array is exhausted first, then simply output the rest of another array

This so-called 2-way merging can be generalized to multi-way merging

## MergeSort

```
public static <E> void mergeSort(E[] data, int size, Comparator<E> comp){
 E[] other = (E[]) new Object[size];
 for (int i=0; i<size; i++) other[i]=data[i];
 mergeSort(data, other, 0, size, comp);
}
private static <E> void mergeSort(E[] data, E[] temp, int low, int high, Comparator<E> comp){
 // sort items from low..high-1 using temp array
 if (high > low+1){
 int mid = (low+high)/2;
 // mid = low of upper 1/2, = high of lower half.
 mergeSort(data, temp, low, mid, comp);
 mergeSort(data, temp, mid, high, comp);
 merge(data, temp, low, mid, high, comp);
 for (int i=low; i<high; i++) data[i]=temp[i];
 }
 // Merge from [low, mid-1] with from [mid, high-1] into to [low, high-1].
 private static <E> void merge(E[] from, E[] to, int low, int mid, int high, Comparator<E> comp){
 int index = low; // where we will put the item into "to"
 int indLeft = low; // index into the lower half of the "from" range
 int indRight = mid; // index into the upper half of the "from" range
 while (indLeft<mid && indRight < high){
 if (comp.compare(from[indLeft], from[indRight]) <= 0)
 to[index++] = from[indLeft++];
 else
 to[index++] = from[indRight++];
 }
 // copy over the remainder. Note only one loop will do anything.
 while (indLeft<mid)
 to[index++] = from[indLeft++];
 while (indRight<high)
 to[index++] = from[indRight++];
 }
}
```

Cost of mergeSort =  $C(n)$

- $C(n) = C(n/2) + C(n/2) +$   
 $= 2 C(n/2) + n$



## Quick sort

partition the array into two parts

partitioning involves the selection of  $a[i]$  where the following conditions are met:

- $a[i]$  is in its final place in the array for some  $i$
- none in  $a[1], \dots, a[i-1]$  is greater than  $a[i]$
- none in  $a[i+1], \dots, a[r]$  is less than  $a[i]$

apply quicksort recursively to each part independently

Divide and Conquer, but does its work in the "split" step

It splits the array into two (possibly unequal) parts:

- choose a "pivot" item
- make sure
  - all items  $<$  pivot are in the left part
  - all items  $>$  pivot are in the right part

Then (recursively) sorts each part

4 3 9 0 8 6  
 $r \quad v$   
 $0 < 6$   
 $4 \ 3 \ 9 \ 7 \ 8 \ 6$   
 $4 \ 3 \ 9 \ 7 \ 8 \ 6$   
 $i=r \quad \text{swap}(9,6)$   
 $4 \ 3 \ 6 \ 7 \ 8 \ 9$

find an 'i'; use  $v = '6'$  to compare  
 use two pointers  $l$  &  $r$ ,  $l$  scan from the left,  
 stop when  $a[l] > v$ ;  
 $r$  scan from right, stop when  $a[r] < v$   
 swap  $a[l]$  &  $a[r]$   
 scan again from where we stop  
 stop again (when  $l \geq r$ );  $i$  is found  
 swap  $a[l]$  with  $v$ ; now every element to the left  
 of 6 is less than 6, every element to the right of  
 6 is greater than 6  
 apply the same process to each partition

right partition sorted.  
 left partition stops scanning, new  $i$  found  
 swap  $a[l]$  with  $v$  (3); left partition sorted  
 Done.

```
public static <E> void quickSort(E[] data, int size, Comparator<E> comp){
 quickSort(data, 0, size, comp);
}

public static <E> void quickSort(E[] data, int low, int high,
 Comparator<E> comp){
 if (high-low < 2) // only one item to sort.
 return;
 else {
 // split into two parts, mid = index of boundary
 int mid = partition(data, low, high, comp);
 quickSort(data, low, mid, comp);
 quickSort(data, mid, high, comp);
 }
}
```

If Quicksort divides the array exactly in half, then:

- $C(n) = n + 2 C(n/2)$
- $= n \log(n) \text{ comparisons} = O(n \log(n))$

## non-linear data structures: trees

BinaryTree tree;

Tree.root =

node 节点

left child / null

right child / null

A binary tree is either:

- empty or a root node together with two binary trees — left / right subtree of root

Different types of tree are used for different things.

- to improve speed
- to improve the use of available memory
- to suit particular problems

术语

- Level: 45, 57, 67, 107 are at the same level
- Depth: the depth of node 60 is 0.
- Leaf Nodes: 没有 children
- Non-Leaf Nodes
- Length: The length of the path from node 60 to 45 is 2.
- Height: the height of the tree is 2. (如果只有一个节点, 为 0. Empty tree 为 -1)
- Siblings: 45 和 57 are siblings

## tree types

Different types are used for different things

- To improve speed
  - To improve the use of available memory
  - To suit particular problems
- But we'll look at three:
- Binary Tree
  - General Tree (n-ary tree)
  - AVL Tree

## useful for

Artificial Intelligence – planning, navigating, games

Representing things:

- Simple file systems
- Class inheritance and composition
- Classification, e.g. taxonomy (the is-a relationship)
- HTML pages
- Parse trees for languages
- Essential in compilers like Java, C# etc.
- 3D graphics (e.g. BSP trees)

Tree represents an efficient 1-dimensional data structure. (T or F?)

A leaf node in a tree has no children. (T or F?)

Binary tree has no ordering upon its sibling nodes. (T or F?)

Name 3 applications for tree.

Relationship among recursive calls can be expressed in what type of tree? recursion tree

## Map

- indexed container
- Associate information with a key
  - key is often a character string
  - info is any information
- E.g. a phone book
  - key is the name of a person or business
  - info is their phone number & address
- Typical Operations on Tables
  - void insert(string key, Object o);
  - object lookup(string key);
  - void remove(string key);
- Alternative implementations include
  - Search Lists:  $O(n)$
  - Binary Search Trees
  - Hash Tables

Maps 可以用 Binary Search Tree 实现.

Binary Search Tree

- 任何节点的左子树中的值 小于 该节点
- 任何节点的右子树中的值 大于 该节点
- 左/右子树也是二叉搜索树

**Binary Search Trees (BST)**

- similar to a linked list, but **two next pointers**
- we call them **left** and **right**
- for each node  $n$ , with key  $k$ 
  - $n \rightarrow \text{left}$  contains only nodes with keys  $< k$
  - $n \rightarrow \text{right}$  contains only nodes with keys  $> k$
- $O(\log N)$**  in average for insert, lookup, and remove

operations can degenerate to  **$O(N)$**  – worst case!

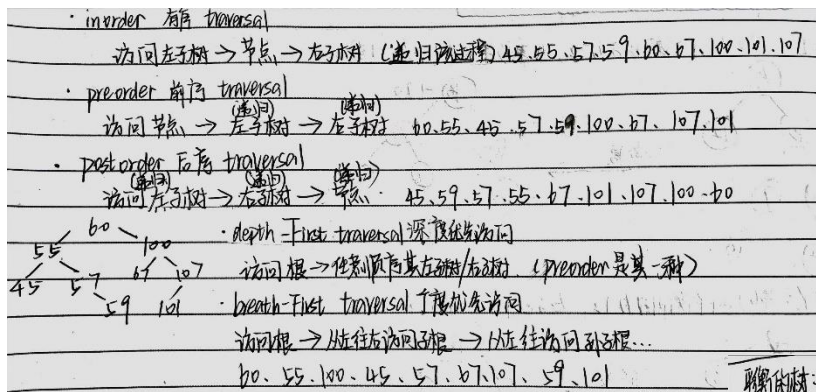
degenerates to a linked list

- when keys are inserted in **ascending order**
  - all keys are to the **right**
- when keys are inserted in **descending order**
  - all keys are to the **left**

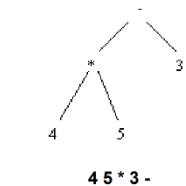
ideal is mid first, then successive middles, etc.

random order also works fairly well

## 遍历 (5 种)



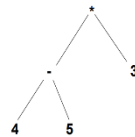
## postorder traversal of the binary expression tree for 4 \* 5 \* 3



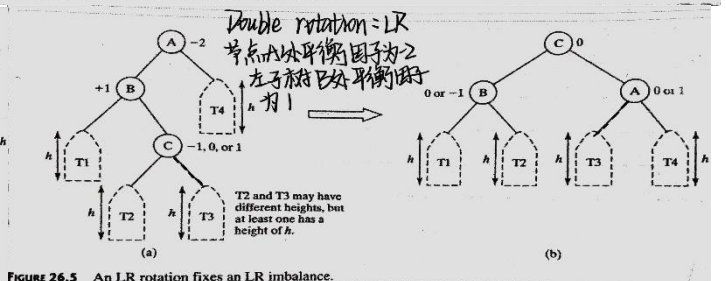
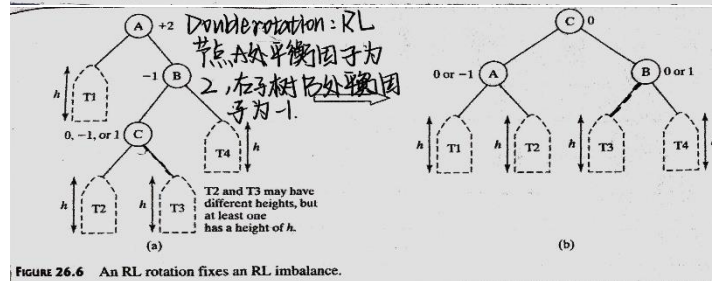
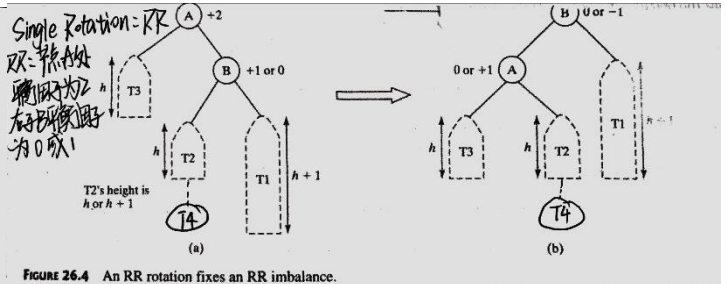
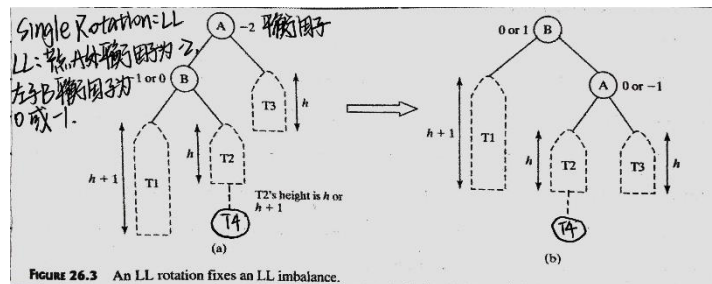
Ex: How do you construct the binary tree for 4\*5\*3 to be evaluated correctly in postorder?

Ex: How would you draw the subtree for (4\*5)\*3 to be evaluated correctly in preorder?

- Correct evaluation in **preorder** should be: \* 4 5 3
- Corresponding tree:



## AVL 4 种 rotation



## Binary Tree ADT

```
public interface BinaryTree<T> {
 BinaryTree<T> getLeft();
 BinaryTree<T> getRight();
 void setLeft(BinaryTree<T> subtree);
 void setRight(BinaryTree<T> subtree);

 BinaryTree<T> find(T value);
 boolean contains(T value);

 boolean isEmpty();
 boolean isLeaf();
 int size();
}
```

## Using BinaryTree (preorder)

```
public static void printAll(BinaryTree<String> tree, String indent){
 System.out.println(indent+ tree.getValue());
 if (tree.getLeft()!=null)
 printAll(tree.getLeft(), indent+" ");
 if (tree.getRight()!=null)
 printAll(tree.getRight(), indent+" ");
}
```

## Constructor

```
public class BinaryTree<V> {
 private V value;
 private BinaryTree<V> left;
 private BinaryTree<V> right;

 /**Creates a new tree with one node
 (a root node) containing the specified value */
 public BinaryTree(V value) {
 this.value = value;
 }
}
```

## Get/set

```
public V getValue() {
 return value;
}

public BinaryTree<V> getLeft() {
 return left;
}

public BinaryTree<V> getRight() {
 return right;
}

public void setValue(V val) {
 value = val;
}

public void setLeft(BinaryTree<V> tree) {
 left = tree;
}

public void setRight(BinaryTree<V> tree) {
 right = tree;
}
```

## isLeaf/find

```
public boolean isLeaf() {
 return (right == null) && (left == null);
}

public BinaryTree<V> find(V val) {
 if (value.equals(val)) return this;
 if (left != null) {
 BinaryTree<V> ans = left.find(val);
 if (ans != null) return ans;
 }
 if (right != null) {
 BinaryTree<V> ans = right.find(val);
 if (ans != null) return ans;
 }
 return null;
}
```



```
public static void main(String[] args)
 BinaryTree<String> myTree = new BinaryTree<String>("Me");
 myTree.setLeft(new BinaryTree<String>("Dad"));
 myTree.setRight(new BinaryTree<String>("Mum"));
 myTree.getLeft().setLeft(new BinaryTree<String>("Gfather"));
 myTree.getLeft().setRight(new BinaryTree<String>("Gmother"));
 myTree.getRight().setLeft(new BinaryTree<String>("Grandpa"));
 myTree.getRight().setRight(new BinaryTree<String>("Grandma"));
 myTree.getRight().getRight().setRight(
 new BinaryTree<String>("GGrandma"));

 BinaryTree<String> gf = myTree.find("Gfather");
 if (gf!=null)
 gf.setRight(new BinaryTree<String>("GGrandma"));
}
```

## Balanced Search Trees

use **rotations** to ensure tree is always 'full'  
prevents degenerative cases mentioned above

truly  $O(\log N)$  worst case for insert,  
lookup, remove

insertion/removal takes more time

but lookup is faster

trickier to code correctly

AVL-一种平衡二叉查找树

| Algorithm | Average     | Worst case  |
|-----------|-------------|-------------|
| Search    | $O(\log n)$ | $O(\log n)$ |
| Insert    | $O(\log n)$ | $O(\log n)$ |
| Delete    | $O(\log n)$ | $O(\log n)$ |
| Space     | $O(n)$      | $O(n)$      |

平衡的树: 每个节点的左右子树差不多高

AVL 是一种平衡二叉查找树 (balanced binary search tree)

每个节点的左右子树的高度差为 0 或 1. (所有 worst case for a search is O(log n))

平衡因子 balance factor: 右子树的高度 - 左子树的高度

一个节点的平衡因子为 -1, 0, 1, 用 0 表示平衡.

如果平衡因子  $\leq 1$ , 节点不重 (left-heavy) 如果  $> 1$  节点右重 (right-heavy).

如果平衡因子  $> 2$  或  $< -2$ , 要重新平衡——通过 rotation.

Rotation doesn't break the existing properties for a search tree. Balanced search



## General Tree Constructor

```
public class GeneralTree<V> {
 private V value;
 private List< GeneralTree<V> > children;

 public GeneralTree(V value) {
 this.value = value;
 this.children = new ArrayList< GeneralTree<V> >();
 }
}
```

## Get

```
public V getValue() {
 return value;
}

public List< GeneralTree<V> > getChildren() {
 return children;
}

public GeneralTree<V> getChild(int i) {
 if (i >= 0 && i < children.size())
 return children.get(i);
 else
 return null;
}
```

## add/find

```
public void addChild(GeneralTree<V> child) {
 children.add(child);
}

public void addChild(int i, GeneralTree<V> child) {
 children.add(i, child);
}

public GeneralTree<V> find(V val) {
 if (value.equals(val)) return this;
 for (GeneralTree<V> child : children) {
 GeneralTree<V> ans = child.find(val);
 if (ans != null) return ans;
 }
 return null;
}
```

## Using General Tree (preorder)

Every algorithm can be expressed iteratively or recursively

In trees, **recursion is normally nicer than iteration** (but not always)

```
public static void printAll(GeneralTree<String> tree, String indent){
 System.out.println(indent+ tree.getValue());
 for(GeneralTree<String> child : tree.getChildren())
 printAll(child, indent+" ");
}

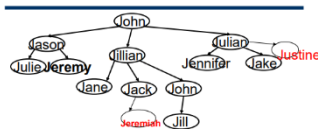
GeneralTree<String> thrd = mytree.getChildren().get(2);
thrd.addChild(new GeneralTree<String>("Justine"));

GeneralTree<String> gc = mytree.find("Jack");
if (gc!=null)
 gc.addChild(new GeneralTree<String>("Jeremiah"));

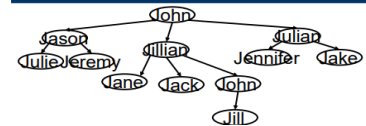
System.out.println ("2nd child of 1st child: "+
 mytree.getChild(0).getChild(1).getValue());
```

Jeremy

mytree = ?



```
public static void main(String[] args){
 GeneralTree<String> mytree = new GeneralTree<String>("John");
 mytree.addChild(new GeneralTree<String>("Jason"));
 mytree.addChild(new GeneralTree<String>("Jillian"));
 mytree.addChild(new GeneralTree<String>("Julian"));
 mytree.getChildren().get(0).addChild(new GeneralTree<String>("Julie"));
 mytree.getChildren().get(0).addChild(new GeneralTree<String>("Jeremy"));
 mytree.getChildren().get(1).addChild(new GeneralTree<String>("Jane"));
 mytree.getChildren().get(1).addChild(new GeneralTree<String>("Jack"));
 mytree.getChildren().get(1).addChild(new GeneralTree<String>("John"));
 mytree.getChildren().get(2).addChild(new GeneralTree<String>("Jennifer"));
 mytree.getChildren().get(2).addChild(new GeneralTree<String>("Jake"));
 mytree.getChildren().get(1).getChild(1).addChild(new
 GeneralTree<String>("Jill"));
}
```



## Hashing and Hash Tables

$O(1)$  in average for insert, lookup, and remove

use an array named  $T$  of capacity  $N$

define a **hash function** that returns an integer  $int H(\text{string key})$

must return an integer between 0 and  $N-1$

store the key and info at  $T[H(\text{key})]$

$H()$  must always return the same integer for a given key

哈希表 **hash** table(key, value) 的做法其实很简单, 就是把Key通过一个固定的算法函数既所谓的哈希函数转换成一个整型数字, 然后将该数字对数组长度进行取余, 取余结果就当作数组的下标, 将value存储在以该数字为下标的数组空间里。

| name     | hash | Hash code | name/number                              |
|----------|------|-----------|------------------------------------------|
| Brecker  | 6    | 0         |                                          |
| Corea    | 2    | 1         |                                          |
| Davis    | 2    | 2         | Corea/46532, Davis/43534, Marsalis/54323 |
| Hancock  | 12   | 3         |                                          |
| Harris   | 12   | 4         |                                          |
| Marsalis | 2    | 5         |                                          |
| Parker   | 8    | 6         | Brecker/37811                            |
|          |      | 7         | Parker/12345                             |
|          |      | 8         |                                          |
|          |      | 9         |                                          |
|          |      | 10        |                                          |
|          |      | 11        |                                          |
|          |      | 12        | Hancock/96562, Harris/32452              |

Hash =  $O(1)$  if no collision

hash function is simply the sum of ASCII codes of characters in a name (considered all in lowercase) computed mod  $N=13$ .

Table size is usually **prime** to avoid bias

overly **large** table size means **wasted space**

overly **small** table size means more **collisions**

## Example of a Modular Hash Function

- $H(k) = k \bmod m$  (or  $k \% m$ )
- message1 = '723416'
- hash function = modulo 11
- Hash value<sub>1</sub> =  $(7+2+3+4+1+6) \bmod 11 = 1$
- message2 = 'test' = ASCII '74', '65', '73', '74'
- Hash value<sub>2</sub> =  $(74+65+73+74) \bmod 11 = 0$
- another hash function example:  $a*k \bmod m$

## for Integer key

Mid-square method

**Squaring** the key value first, and then takes out the **middle  $r$  bits** of the result, giving a value in the range 0 to  $2^r-1$ .

This works well because most or all bits of the key value contribute to the result

```
4567
4567
31969
27402
22835
18268
20857489
4567
```

## What is a Hash Function?

A hash function  $H$  is a transformation that

- takes a variable-size input  $k$  and
- returns a fixed-size string (or int), which is called the **hash value**  $h$  (that is,  $h = H(k)$ )

a good hash function has the following characteristics:

- avoids collisions
- spreads keys evenly in the array
- inexpensive to compute - must be  $O(1)$

## For string key

This function takes a string as input. It processes the string 4 bytes at a time, and interprets each of the four-byte chunks as a single long integer value. The integer values for the 4-byte chunks are added together. The resulting sum is converted to the range 0 to  $M-1$  using the modulus operator. There is nothing special about using 4 characters at a time. Other choices could be made.

- must be careful to cover range from 0 through capacity-1
- some poor choices
  - summing all the ASCII codes
  - multiplying the ASCII codes
- important insight
  - letters and digits fall in range 0101 and 0172 octal
  - so all useful information is in lowest 6 bits

hash(s) is  $O(1)$

解决哈希冲突

开放地址法: 当在数组末端时, 开放空地址给冲突, 直到找到空单元

开放地址法的变种: 线性探测, 二次探测和双散列

Linear probing: 线性: 如果  $k \% N$  中有 value, 就  $(k+1) \% N$ ; 再  $(k+2) \% N$  直到找到空位

缺点: clustering problem 聚集问题, search 时间变长

Quadratic probing: 二次:  $(k+j^2) \% N$  0, 1, 4, 9, ...

缺点: 二次聚集问题 / 表和满时不一定能找到空位 (线性没有这个问题)

double hashing

双散列:  $k \% N, (k+h_1(\text{key})) \% N, (k+h_2(\text{key})) \% N, \dots$

$h$  是函数,  $h(\text{key}) = k \% N$   $h'$  是次函数

## Graph theory

**Definition:** A graph  $G$  consists of: (1) a nonempty set of vertices (点,  $V$ )  
(2) a set of edges that connect the vertices ( $E$ )  
一个 graph 的定义  $G = (V, E)$  图中  $V$  的个数称为图的阶数, 用  $|V|$  表示

**adjacent:** 两个顶点由边连接, 则这两个顶点是 adjacent 相邻的

**neighbors:** 相邻顶点称为邻点

**incident:** 连接顶点的边 is incident 关联到 the vertex.

**Multiple edges:** 连接同一对顶点的两条或多条边是 multiple edges

**loop:** 一个顶点连接自身的边称为 loop 环

**Simple graphs:** 不含 loop 和 multiple 的图

**walk:** A sequence of edges in the form of  $v_0v_1, v_1v_2, v_2v_3$  is a walk from  $v_0$  to  $v_3$   
**trail:** 这些边都是不同的, the walk is called trail.

**path:** 顶点不同的 trail

**closed:** a walk or trail is closed if  $s=k$ .

**cycle/circuit:** a closed walk 中的所有顶点都不相同除  $v_0$  和  $v_k$ .

**length:** the number of edges in a walk.

**Tree:** A tree is a connected graph with no cycles

**forest:** a graph with no cycles and it may be connected or not.

**Tree 性质:** 1. 从任一顶点可以通向其他顶点.

(at least two) 2. 从树中去掉任一条 edge, 会变成两个 components, 每个 component 都是树.

**Spanning tree:** a spanning tree of a graph  $G$  is:

- a tree
- a spanning subgraph of  $G$  (生成树有  $n$  个顶点和  $n-1$  条边)

**weighted graphs 加权图**

有一个数字与每条边对应的加权边

这个数字利用距离表示, 度量标准

**digraph 有向图**

可以用有向图模拟关系

绘制 underlying graph of digraph

**Degree 度:** 连接顶点  $V$  的边数  $d(V)$  loop 要算两次

**degree sequence 度序列:** 由顶点的度按递增顺序排列组合, 必要时重复

一个简单图的所有顶点的度  $d(V)$  的值的和是边数的两倍:  $\sum d(V_i) = 2|E|$

有向度的顶点: 入度 in-degree  $d^-(V)$  出度 out-degree  $d^+(V)$

$\sum d^-(V_i) = |E|$   $\sum d^+(V_i) = |E|$   $A$  是  $n \times n$  邻接数

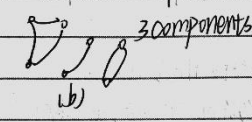
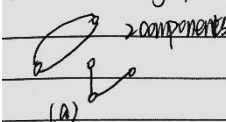
子图  $H = (V_1, E_1)$

**subgraph of graph  $G(V, E)$ :**  $V_1 \subseteq V, E_1 \subseteq E$ .

**spanning subgraph 支撑子图**  $= V_1 = V$

**Connected graph:** 有一条从任意顶点  $v_i$  到任何其他顶点的 path.

A disconnected graph is said to be made up of components.



**minimum spanning tree:** 给定一个加权图, 最小生成树是其中代价最小的生成树.

应用: crypton planning; communications

**The greedy algorithm:** 从任一顶点开始

添加最便宜边 (不添顶点), 重复这一步

大概一半化简其中之一.

**Why data structures?** primitive types, like **boolean, integers and strings**, and simple arrays are not enough since the memory model of a computer is simply an array of integers • But, this model: 1. is conceptually inadequate 不充分的 & low level, since information is usually expressed in the form of highly structured data 2. makes it difficult to describe complex algorithms, since the basic operations are too primitive

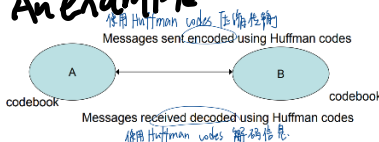
**Software quality** • Whenever develop a software system, should create good software. 1. 软件的质量衡量方式 – **correctness, – efficiency.**

main principles that help developing good software: • **abstraction, information hiding, encapsulation.** • important design issue related to DS that help producing good software: the choice between **static** and **dynamic structures.**

**Abstraction** • As a **process**, it denotes the extracting of the **essential details** about an item, or a group of items, while ignoring the **nonessential** details. • As an **entity**, abstraction denotes a **model**, a view, or some representation for an actual item which leaves out some of the details of the item. • Abstraction dictates that **some information is more important than other information**, but does **not provide a specific mechanism for handling** the unimportant information.

• In the context of **software development**: • The aim of **data abstraction** is to identify which details of how data is stored and can be manipulated are important and which are not • The aim of **procedural abstraction** is to identify which details of how a task is accomplished are important and which are not

### An example



**Communication based on Huffman coding** • an effective way of encoding(+decoding) textual (or non-textual) data. used in communication, e.g., source coding • A large information system • The Huffman encoding software uses

**priority queue** to accomplish its tasks. • Important points: – The description abstracts from all details on how the priority queue and its operations are implemented. **Key ideas** • Based on the **frequency** of occurrence of a data item

(pixel in images, alphabet in texts). • The principle is to use a **smaller number of bits** to encode the data that **occurs more frequently** • Codes are stored in a **Code Book** which may be constructed for each image (alphabet) or a set of images • In all cases the code book+encoded data must be transmitted to enable decoding.

**Information hiding (IH)** • is the principle that users of a module need to know **only the essential details of this module** (as identified by abstraction) • abstraction can identify details of a module 对用户来说哪些重要哪些不重要 • IH keep all unimportant details secret from the user and try to prevent him from making use any unimportant details • The important details of a module that a **user needs to know form the specification** • So, IH means that **modules are used via their specifications, not their implementations.** • IH hides the internal data or information from direct manipulation. • IH is related to privacy, security

Example: • Cars provide an example of this in how they interface with drivers. • They present a **standard interface O-O: publicizes** services it is capable of providing, or its method headers, **hides** the internal details both of **how** it performs these services and of the data (attributes & structures) that it maintains in order to support these services.